UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
PROFESSOR DR.: FÁBIO LUIZ LEITE JÚNIOR

ADRIELLY CARLA FERREIRA DE MELO - 2023208510013

MARIA EDUARDA DA NOBREGA - 2024108510039

LINK DO REPOSITÓRIO: https://github.com/eduardanb/sudoku-solver-taal.git

PROJETO DE ANÁLISE DE ALGORITMOS: SUDOKU SOLVER, RELATÓRIO COMPARATIVO.

CAMPINA GRANDE - PB

2026

SUMÁRIO

1. INTRODUÇÃO **4**
1.1. Contextualização do Problema 4
1.2. Objetivos 4
1.2.1. Geral 4
1.2.2. Específico 4
2. METODOLOGIA E IMPLEMENTAÇÃO **4**
2.1. Estrutura do Sistema Desenvolvido 4
2.1.1. Pacote Experiment 4
2.1.2. Pacote IO 4
2.1.3. Pacote Model 4
2.1.4. Pacote Solver e Subpacotes 4
2.1.5. Pacotes Heuristics e Utils 4
2.1.6. Pacote Metrics 4
2.2. Formato de Dados de Entrada e Saída 4
2.2.1. Arquivos de entrada 4
2.2.2. Saída em Console 4
2.2.3. Arquivos de Saída 4
2.3. Métricas e Avaliação 4
2.3.1. Métricas de desempenho coletadas 4
2.3.2. Protocolo de experimentos em lote 4
2.3.3. Variação da corretude das soluções 4
3. ALGORITMOS IMPLEMENTADOS **4**
3.1. Backtracking 4
3.1.1. Análise de Corretude 4
3.1.2. Análise de Desempenho 4
3.1.2.1. Espacial 4
3.1.2.2. Temporal 4
3.2. Branch and Bound 4
3.2.1. Análise de Corretude 4
3.2.2. Análise de Desempenho 4
3.2.2.1. Espacial 4
3.2.2.2. Temporal 4
3.3. Estratégia Gulosa (Greedy) 4
3.3.1. Análise de Corretude 4
3.3.2. Análise de Desempenho 4
3.3.2.1. Espacial 5
3.3.2.2. Temporal 5
3.4. Programação Dinâmica 5
3.4.1. Análise de Corretude 5
3.4.2. Análise de Desempenho 5

3.4.2.1. Espacial 5
3.4.2.2. Temporal 5
4. EXPERIMENTOS E RESULTADOS 5
4.1. Instâncias de Testes 5
4.2. Instâncias Obtidas 5
5. ANÁLISE E DISCUSSÃO 5
5.1. Comparação das Estratégias 5
5.1.1. Visão Geral das Técnicas 5
5.1.2. Complexidade Computacional 5
5.1.3. Eficiência e Desempenho 5
5.1.4. Qualidade da Solução 5
5.1.5. Aplicabilidade 5
5.1.6. Vantagens e Desvantagens 5
5.1.7. Síntese Comparativa 5
6. CONCLUSÃO 5
7. REFERÊNCIAS 5

1. INTRODUÇÃO

No curso de Técnicas de Análise de Algoritmos, foram sugeridas atividades em grupos com a finalidade de criar soluções computacionais usando as técnicas de algoritmos ensinadas nas aulas. Cada equipe deveria escolher um dos temas indicados pelo professor e desenvolver uma solução com base nos métodos de design de algoritmos discutidos durante o curso.

A opção que nossa equipe fez foi o SudokuSolver, que se refere ao tema 13 da lista de sugestões. A tarefa envolve criar um sistema que consiga resolver um tabuleiro de Sudoku automaticamente, utilizando as estratégias algorítmicas apropriadas para identificar uma solução válida.

Para solucionar o problema, diversas técnicas de design de algoritmos foram consideradas, incluindo Backtracking, Branch and Bound, Greedy e Programação Dinâmica, permitindo a avaliação de suas utilizações e eficiência na tarefa de resolver o Sudoku.

1.1. Contextualização do Problema

O Sudoku é um jogo de lógica antigo, amplamente conhecido que consiste em preencher um tabuleiro de 9x9 células, dividido em nove sub-grades de 3x3, com números de 1 a 9. O objetivo do jogo é completar o tabuleiro respeitando determinadas restrições: cada número deve aparecer apenas uma vez em cada linha, em cada coluna e em cada sub-grade 3x3.

No contexto computacional, o Sudoku pode ser tratado como um problema de busca com restrições, no qual um algoritmo deve encontrar valores válidos para as células vazias de um tabuleiro parcialmente preenchido. Nesse problema, algumas posições já possuem números definidos, enquanto outras estão vazias e precisam ser completadas de forma que todas as regras do jogo sejam satisfeitas.

No problema abordado neste projeto, o tabuleiro de entrada é representado por uma matriz 9x9 de números inteiros, onde o valor 0 representa uma célula vazia que precisa ser preenchida. O algoritmo desenvolvido deve analisar o tabuleiro inicial e preencher os espaços vazios com números de 1 a 9 de forma que nenhuma das restrições do Sudoku seja violada.

A entrada do programa consiste em 9 linhas contendo 9 números inteiros cada, representando o estado inicial do tabuleiro. O algoritmo deve então produzir como saída o tabuleiro completo solucionado, sem quebrar nenhuma regra do jogo.

Do ponto de vista da análise de algoritmos, o problema de resolver Sudoku é interessante por envolver espaços de busca potencialmente grandes, o que torna necessária a utilização de técnicas algorítmicas eficientes para explorar as possíveis combinações de preenchimento do tabuleiro. Dessa forma, métodos de projeto de algoritmos como Backtracking, Branch and Bound, Greedy e Dynamic Programming podem ser investigados e comparados quanto ao seu desempenho na resolução do problema.

Além da obtenção de uma solução válida, também é possível analisar métricas importantes relacionadas à execução do algoritmo, como tempo de resolução, número de nós visitados durante a busca e tamanho da árvore de busca gerada, permitindo avaliar a eficiência das estratégias utilizadas.

1.2. Objetivos

Nesta seção, são apresentados os objetivos gerais e específicos da presente atividade.

1.2.1. Geral

O objetivo geral do presente trabalho é desenvolver um algoritmo capaz de resolver automaticamente um tabuleiro de Sudoku 9x9 parcialmente preenchido, entre os níveis fácil, médio e difícil, aplicando técnicas de projeto e análise de algoritmos estudados na disciplina.

1.2.2. Específico

Os objetivos específicos focam entre implementar um algoritmo capaz de preencher corretamente as células vazias de um tabuleiro de Sudoku, respeitando todas as regras do jogo. Para isso, são utilizadas diferentes técnicas de projeto de algoritmos, como Backtracking, Branch and Bound, Greedy e Dynamic

Programming, com o intuito de investigar suas aplicações na resolução do problema.

Além da implementação das abordagens propostas, busca-se avaliar o desempenho de cada técnica por meio de métricas como tempo de execução, número de nós visitados e tamanho da árvore de busca gerada durante o processo de resolução. Esses indicadores permitem analisar o comportamento dos algoritmos e compreender sua eficiência computacional.

Também são comparados os resultados obtidos a partir de diferentes instâncias do problema, considerando tabuleiros classificados como fáceis, médios e difíceis. Essa comparação possibilita observar como cada técnica se comporta diante de diferentes níveis de complexidade do Sudoku.

Por fim, é realizada uma análise das técnicas algorítmicas utilizadas, identificando suas principais vantagens e limitações na resolução do problema, bem como discutindo sua adequação e desempenho para esse tipo de aplicação.

2. METODOLOGIA E IMPLEMENTAÇÃO

Nesta seção, são descritos a estrutura do sistema desenvolvido, o formato dos dados de entrada e saída, bem como as métricas utilizadas para avaliação.

2.1. Estrutura do Sistema Desenvolvido

O sistema foi projetado seguindo o princípio de separação de responsabilidades, distribuindo as funcionalidades em pacotes coesos e com acoplamento reduzido. A estrutura de diretórios segue a convenção Maven, com código-fonte em *src/main/java*, recursos estáticos em *src/main/resources* e testes automatizados em *src/test/java*. A organização interna do código é distribuída em seis pacotes principais, cada um com responsabilidade bem delimitada, conforme descrito a seguir.

2.1.1. Pacote Experiment

O pacote br.edu.sudoku.experiment concentra o ponto de entrada interativo do sistema, representado pela classe ExperimentRunner. Esta classe implementa um laço de interação via terminal que permite ao usuário selecionar, a cada execução, o algoritmo desejado (Backtracking, Branch and Bound, Greedy ou Programação Dinâmica) e o nível de dificuldade do jogo (fácil, médio ou difícil). Internamente, o ExperimentRunner é responsável por orquestrar o fluxo completo de uma execução: carregamento do tabuleiro, instanciação do solver, temporização da resolução e exibição dos resultados no console, incluindo o tabuleiro inicial, o tabuleiro resolvido e as métricas de desempenho coletadas.

Além do ExperimentRunner, o pacote também abriga a classe PerformanceComparisonTest, voltada à execução em lote: ela percorre automaticamente todas as combinações de algoritmo e dificuldade, repete cada experimento dez vezes e calcula médias e desvios padrão, provendo uma base estatística robusta para a comparação de desempenho.

2.1.2. Pacote IO

O pacote br.edu.sudoku.io encapsula toda a lógica de entrada e saída do sistema. A classe SudokuReader é responsável por carregar um tabuleiro a partir de um arquivo de texto simples localizado no classpath, utilizando ClassLoader.getResourceAsStream para garantir compatibilidade com a estrutura de recursos do Maven. O arquivo é lido linha a linha com BufferedReader.readLine(), os valores são separados por espaço e o resultado é encapsulado em um objeto SudokuBoard.

A classe SudokuWriter oferece dois métodos: printBoard, que delega a exibição formatada ao próprio modelo, e writeToFile, que persiste a solução em um arquivo de texto com o mesmo formato da entrada. O nome do arquivo de saída é composto dinamicamente a partir do nível de dificuldade e do algoritmo utilizado, por exemplo, sudoku_medio_backtracking.txt, favorecendo a rastreabilidade dos resultados.

2.1.3. Pacote Model

O pacote br.edu.sudoku.model contém a classe SudokuBoard, que representa o estado do tabuleiro em um dado momento. Internamente, o tabuleiro é mantido como uma matriz bidimensional int[9][9], onde o valor 0 indica uma célula ainda não preenchida. Adicionalmente, a classe mantém uma matriz booleana boolean[9][9] que registra, no momento da criação do objeto, quais células possuíam valores originalmente fixos (as pistas do jogo). Essa distinção é utilizada na impressão do tabuleiro: células fixas são exibidas em azul ciano via código de escape ANSI (\u001B[36m), facilitando a diferenciação visual entre as pistas fornecidas e os valores inseridos pelo solver durante a resolução.

A separação entre células fixas e células preenchidas pelo solver é especialmente importante para a integridade da busca: os algoritmos nunca tentam sobrescrever células fixas, pois operam apenas sobre posições onde board.get(linha, coluna) == 0.

2.1.4. Pacote Solver e Subpacotes

O pacote br.edu.sudoku.solver define a interface central SudokuSolver, que declara o contrato único compartilhado por todos os algoritmos: o método boolean solve(SudokuBoard tabuleiro, Metrics metricas). Esse contrato permite que o ExperimentRunner trabalhe de forma polimórfica com qualquer implementação de solver, sem precisar conhecer os detalhes internos de cada algoritmo. Os algoritmos são organizados em quatro subpacotes independentes, cada um contendo uma interface específica e sua respectiva classe de implementação. A tabela a seguir resume os quatro algoritmos propostos:

Algorimo	Estratégia de seleção	Heurística	Status
Backtrakcing	Linear (linha x coluna)	Nenhuma	Implementado
Branch and Bound	Célula mais restrita	MRV + Bound de inviabilidade	Implementado
Greedy	Célula mais restrita	MRV + LCV	Implementado
Prog. Dinâmica	Célula mais restrita	MRV + Memoização	Implementado

Tabela 1 - Algoritmos de resolução implementados no sistema.

O Backtracking (BacktrackingSolver) implementa a busca por retrocesso clássica: percorre as células vazias em ordem linear (linha por linha, coluna por coluna), tenta inserir os valores de 1 a 9 em cada posição e retrocede ao detectar uma violação de restrição. A cada chamada recursiva, o nó visitado é contabilizado pela instância de Metrics; cada retrocesso incrementa o contador de backtracks. O algoritmo suporta, opcionalmente, um modo de visualização passo a passo, ativado pelo parâmetro booleano visualizar no construtor, que exibe o estado do tabuleiro a

cada atribuição com um atraso de 150 ms, tornando o processo de busca observável em tempo real.

O Branch and Bound (BranchAndBoundSolver) incorpora a heurística MRV (Minimum Remaining Values) para guiar a seleção de variáveis. Em vez de escolher a próxima célula vazia em ordem linear, o algoritmo consulta a classe MRVHeuristic, que varre todas as células vazias e retorna aquela com o menor número de valores candidatos válidos. Ao priorizar as células mais restritas, a heurística reduz significativamente o espaço de busca explorado, detectando conflitos mais cedo na árvore de decisão. O que distingue este algoritmo do backtracking simples é a função de bound: após cada atribuição, o método calcularBound percorre todas as células vazias restantes e conta quantas ficaram sem nenhum candidato válido. Se esse contador for maior que zero, o ramo é podado imediatamente, pois nenhuma solução válida pode ser obtida a partir daquele estado. Como otimização adicional, quando uma célula com apenas um candidato é encontrada durante a seleção MRV, o método retorna imediatamente sem varrer o restante do tabuleiro.

O Greedy (GreedySolver) combina duas heurísticas para guiar a busca. A seleção de variável segue o critério MRV, priorizando sempre a célula vazia com menor domínio, com retorno antecipado ao encontrar domínio igual a 1. A ordenação dos candidatos segue o critério LCV (Least Constraining Value): para cada valor candidato, o algoritmo o atribui temporariamente à célula e soma os candidatos restantes de todas as células vizias na mesma linha, coluna e subquadrado 3×3; valores que menos reduzem as opções dos vizinhos são tentados primeiro. A ordenação é feita por selection sort sobre os custos calculados. Quando nenhum candidato leva à solução, um único backtrack é registrado para a célula, e a busca retrocede.

A Programação Dinâmica (DynamicProgrammingSolver) aplica memoização para evitar a reexploração de estados já provados como sem solução. A cada chamada recursiva, o estado atual do tabuleiro é serializado em uma string de 81 caracteres, onde cada posição representa o valor de uma célula, e verificado contra um HashSet chamado estadosMortos. Se o estado já estiver registrado, a chamada retorna imediatamente sem explorar o ramo, economizando trabalho redundante. A seleção de variável também segue o critério MRV, com retorno antecipado para

domínio igual a 1. Quando nenhum valor funciona para a célula selecionada, o estado atual é inserido no conjunto de estados mortos e um único backtrack é registrado, evitando que o mesmo estado seja processado novamente em outros ramos da árvore de busca.

2.1.5. Pacotes Heuristics e Utils

O pacote br.edu.sudoku.heuristics define a interface VariableOrderingHeuristic, com o método int[] selecionarCelula(SudokuBoard tabuleiro), e a implementação MRVHeuristic. O método retorna um array de dois elementos com as coordenadas [linha, coluna] da célula escolhida. Internamente, para cada célula vazia, o método auxiliar obterCandidatos constrói um array de até 10 posições: a posição de índice 0 armazena a contagem de candidatos válidos, e as posições 1 a 9 armazenam os próprios valores candidatos. Essa estrutura é reutilizada tanto na heurística quanto no BranchAndBoundSolver, evitando alocações redundantes.

O pacote br.edu.sudoku.utils contém a classe SudokuValidator, cujo método estático isValid(SudokuBoard board, int linha, int col, int num) verifica, em tempo O(n), se a inserção de um valor em uma célula viola alguma das três restrições do Sudoku: unicidade na linha, unicidade na coluna e unicidade na subgrade 3×3. A subgrade relevante é determinada pelas fórmulas (linha / 3) * 3 e (col / 3) * 3, que mapeiam qualquer célula para o canto superior esquerdo de seu bloco. Valores fora do intervalo [1, 9] são imediatamente rejeitados, antes mesmo de qualquer verificação estrutural.

2.1.6. Pacote Metrics

O pacote br.edu.sudoku.metrics encapsula a coleta de dados de desempenho por meio da classe Metrics. Cada execução de solver recebe uma instância isolada, garantindo que os contadores não sejam compartilhados entre experimentos distintos. A classe expõe dois contadores incrementais, visitedNodes e backtracks,

ambos do tipo long para suportar instâncias de alta dificuldade que podem gerar volumes elevados de chamadas recursivas sem risco de overflow.

2.2. Formato de Dados de Entrada e Saída

O sistema opera sobre arquivos de texto estruturados que definem o estado inicial do tabuleiro e registram a solução obtida ao término da execução.

2.2.1. Arquivos de entrada

Os tabuleiros de entrada são armazenados como arquivos de texto simples (.txt) no diretório *src/main/resources/sudokus/*. Cada arquivo representa um tabuleiro 9×9 cujas linhas correspondem diretamente às linhas do jogo: nove valores numéricos inteiros separados por espaço simples. Células sem valor predefinido são representadas pelo inteiro 0. O sistema disponibiliza três instâncias com graus de dificuldade distintos:

- sudoku_facil.txt: jogo com elevado número de pistas, exigindo menor exploração.
- sudoku_medio.txt: jogo com número intermediário de pistas.
- sudoku_difícil.txt: jogo com poucas pistas, exigindo uma busca mais extensa.

A seguir, apresenta-se um exemplo do formato de entrada:

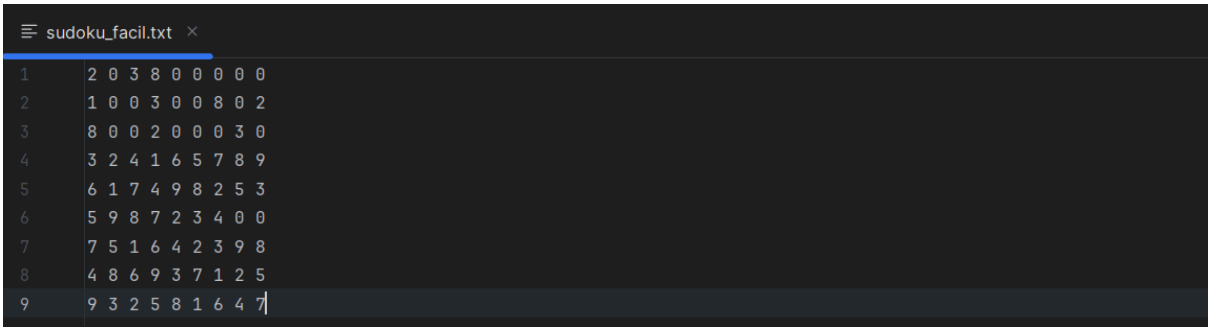

Figura 1 - Exemplo de arquivo de entrada, nível fácil.

Internamente, a leitura é realizada pela classe SudokuReader, que utiliza ClassLoader.getResourceAsStream(path) para localizar o arquivo no classpath,

percorre as nove linhas com BufferedReader.readLine() e converte cada token em inteiro via Integer.parseInt. O resultado é encapsulado em um objeto SudokuBoard, que registra automaticamente quais posições eram não-nulas como células fixas.

2.2.2. Saída em Console

Durante a execução interativa, o sistema exibe no console o tabuleiro inicial e, ao término da resolução, o tabuleiro completo, por meio do método SudokuBoard.printBoard(). A formatação adota separadores verticais e horizontais para demarcar os nove blocos 3×3 do jogo. Células cujos valores fazem parte das pistas originais são exibidas em azul ciano (código ANSI \u001B[36m), enquanto os valores inseridos pelo solver são exibidos com a cor padrão do terminal. Ao término da resolução, são apresentadas as métricas de desempenho: tempo de execução em milissegundos, total de nós visitados e número de backtracks.

A seguir, apresenta-se uma amostra do formato da saída no console:

Figura 2 - Início de execução.

```
=== Sudoku Solver (Branch and Bound) ===
Dificuldade: DIFICIL
Passo: 66
Tentando colocar 9 em (6,7)

+-------+-------+-------+
| 4 7 9 | 8 6 5 | 2 3 1 |
| 5 2 6 | 3 7 1 | 8 4 9 |
| 8 3 1 | 4 2 9 | 5 6 7 |
+-------+-------+-------+
| 6 9 8 | 5 3 7 | 1 2 4 |
| 7 4 2 | 6 1 8 | 9 5 3 |
| 1 5 3 | 2 9 4 | 7 8 6 |
+-------+-------+-------+
| 3 1 4 | 7 5 2 | 6 9 8 |
| 9 6 5 | 1 8 3 | 4 7 2 |
| 2 8 7 | 9 4 6 | 3 1 5 |
+-------+-------+-------+
```

Figura 3 - Exemplo de finalização de Sudoku, técnica Branch and Bound, nível difícil.

```
Sudoku resolvido:

+-------+-------+-------+
| 4 7 9 | 8 6 5 | 2 3 1 |
| 5 2 6 | 3 7 1 | 8 4 9 |
| 8 3 1 | 4 2 9 | 5 6 7 |
+-------+-------+-------+
| 6 9 8 | 5 3 7 | 1 2 4 |
| 7 4 2 | 6 1 8 | 9 5 3 |
| 1 5 3 | 2 9 4 | 7 8 6 |
+-------+-------+-------+
| 3 1 4 | 7 5 2 | 6 9 8 |
| 9 6 5 | 1 8 3 | 4 7 2 |
| 2 8 7 | 9 4 6 | 3 1 5 |
+-------+-------+-------+

================ RESULTADOS ================
Tempo de execução: 10135 ms
Nós visitados: 52
Backtracks: 14
============================================
```

Figura 4 - Amostra do sudoku resolvido no terminal com as métricas.

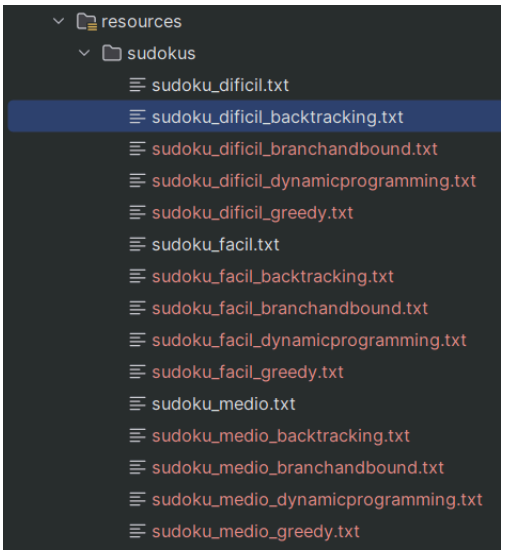

Figura 5 - Após a execução, o Sudoku resolvido fica na pasta sudokus.

2.2.3. Arquivos de Saída

Após a resolução bem-sucedida, o tabuleiro completo é gravado em arquivo de texto no mesmo diretório de recursos, com nome composto dinamicamente no padrão sudoku_<dificuldade>_<algoritmo>.txt, por exemplo, sudoku_medio_backtracking.txt. O formato de saída é idêntico ao de entrada: nove linhas de nove valores separados por espaço. Essa decisão garante interoperabilidade total entre os arquivos do sistema: qualquer saída pode ser reutilizada como entrada sem nenhuma conversão adicional, facilitando a validação manual das soluções e a comparação direta entre os resultados de diferentes algoritmos sobre o mesmo tabuleiro.

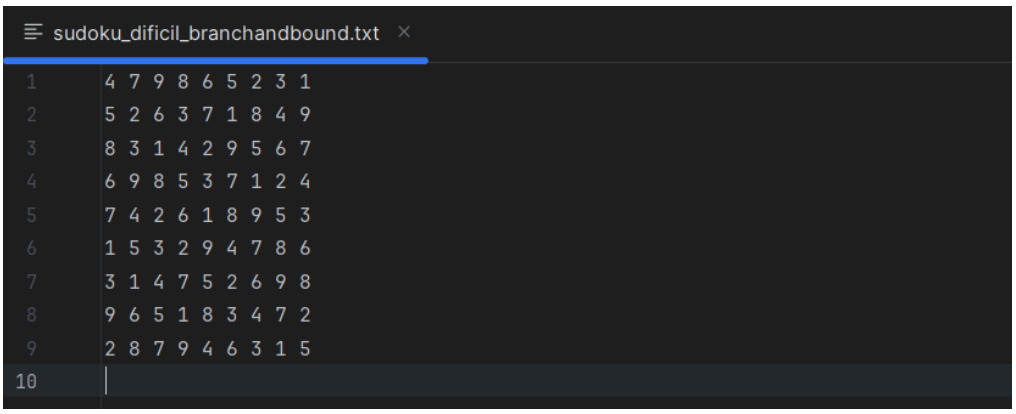

Figura 5 - Saída em arquivo .txt do Sudoku nível difícil utilizando Branch and Bound.

2.3. Métricas e Avaliação

A avaliação comparativa dos algoritmos foi estruturada em três dimensões complementares: eficiência de busca, custo temporal e corretude da solução.

2.3.1. Métricas de desempenho coletadas

Todas as métricas de busca são coletadas durante a execução por meio da classe Metrics, cuja instância é passada por parâmetro a cada chamada do método solve. Isso garante isolamento total dos contadores por execução, sem efeitos colaterais entre experimentos.

Métrica	Unidade	Coletada
Nós visitados	Contagem inteira	Metrics.getVisitedNodes()
Backtracks	Contagem inteira	Metrics.getBacktracks()
Tempo de execução	Milissegundos (ms)	System.nanoTime()
Desvio padrão	Milissegundos (ms)	PerformanceComparisonTest

Tabela 2 - Métricas coletadas pelo sistema.

2.3.2. Protocolo de experimentos em lote

Para reduzir o impacto de variações do ambiente de execução, como aquecimento da JVM, coleta de lixo e escalonamento de threads, os experimentos comparativos são conduzidos pela classe PerformanceComparisonTest, seguindo o protocolo abaixo:

- Cada combinação de algoritmo e nível de dificuldade é executada dez vezes consecutivas, utilizando a constante EXECUCOES = 10.
- Para cada execução, o tabuleiro é recarregado a partir do arquivo, garantindo estado inicial idêntico entre as repetições.
- Uma nova instância de Metrics é criada a cada execução, evitando o acúmulo de contagens.
- Ao final, são calculadas a média aritmética e o desvio padrão do tempo de execução, a média de nós visitados e a média de backtracks.

O desvio padrão do tempo de execução funciona como indicador de estabilidade do algoritmo. Valores baixos indicam comportamento consistente entre execuções, enquanto valores altos sugerem sensibilidade a fatores externos, como coleta de lixo ou variações de carga da JVM.

2.3.3. Variação da corretude das soluções

A validade das soluções produzidas por cada algoritmo é verificada automaticamente pelos testes JUnit 5 implementados em BacktrackingSolverTest, BranchAndBoundSolverTest e SolverComparisonTest. O método auxiliar tabuleiroCompletoEValido percorre todas as 81 células do tabuleiro solução e, para cada uma, verifica duas condições: (1) o valor atribuído está no intervalo [1, 9]; e (2) esse valor é consistente com as restrições do jogo, utilizando

SudokuValidator.isValid após temporariamente zerar a célula para evitar falso conflito da célula consigo mesma.

Os testes unitários em SudokuValidatorTest cobrem individualmente os seis cenários relevantes de validação: inserção válida, conflito de linha, conflito de coluna, conflito de subgrade 3×3, valor zero e valor acima de 9. Essa cobertura garante que a função de validação, base de todos os solvers, esteja correta antes que qualquer algoritmo seja avaliado. Os níveis de dificuldade e os algoritmos a serem testados são configurados dinamicamente por meio de System.setProperty, permitindo selecionar qualquer combinação sem necessidade de recompilação.

3. ALGORITMOS IMPLEMENTADOS

Esta seção apresenta uma análise detalhada dos algoritmos de resolução implementados no sistema, abordando seus fundamentos teóricos, decisões de implementação, corretude formal e complexidade computacional. Para cada algoritmo, são examinadas separadamente a complexidade espacial e a complexidade temporal, considerando tanto o caso geral quanto o comportamento esperado no contexto específico do Sudoku 9×9.

3.1. Backtracking

O Backtracking é uma técnica algorítmica de busca exaustiva com poda, amplamente utilizada na resolução de problemas de satisfação de restrições. A ideia central consiste em construir a solução incrementalmente, atribuindo valores a variáveis uma a uma e abandonando imediatamente qualquer atribuição parcial que viole alguma restrição, sem precisar explorar todas as extensões possíveis daquela atribuição. No contexto do Sudoku, cada célula vazia corresponde a uma variável, e as restrições são as regras de unicidade por linha, coluna e subgrade 3×3.

A implementação do sistema segue esse paradigma de forma direta. A classe BacktrackingSolver percorre as células vazias em ordem linear, linha por linha da esquerda para a direita, tentando inserir os valores de 1 a 9 em cada posição.

Quando nenhum valor é válido para a célula atual, a última atribuição é desfeita e o algoritmo retrocede para a célula anterior. Esse processo se repete recursivamente até que todas as células estejam preenchidas, indicando que a solução foi encontrada, ou até que o espaço de busca se esgote completamente, caracterizando um jogo sem solução.

Um aspecto relevante da implementação é o suporte ao modo de visualização passo a passo, ativado pelo parâmetro booleano visualizar no construtor. Quando ativo, o algoritmo exibe o estado do tabuleiro a cada atribuição e a cada retrocesso, com um atraso de 150 ms entre os passos por meio de Thread.sleep. Esse recurso é desativado nos testes automatizados e nos experimentos em lote, onde a instância é criada com new BacktrackingSolver(false), eliminando o overhead de I/O e de temporização do ciclo de medição. O nome do nível de dificuldade é recuperado dinamicamente por meio de System.getProperty("difficulty"), permitindo que tanto o ExperimentRunner quanto os testes JUnit configurem o rótulo exibido sem a necessidade de repassar o parâmetro pela cadeia de chamadas recursivas.

3.1.1. Análise de Corretude

A corretude de um algoritmo de busca com retrocesso pode ser demonstrada por três propriedades complementares: completude, validade e terminação.

O algoritmo é completo para instâncias de Sudoku bem formadas, pois o espaço de busca do Sudoku 9×9 é finito e limitado, com no máximo 9^{81} estados possíveis, embora a grande maioria seja eliminada pelas restrições logo nas primeiras atribuições. A cada nível de recursão, o algoritmo experimenta todos os valores candidatos para a célula atual, de 1 a 9, antes de retornar false. Se alguma combinação de valores leva a uma solução, ela será necessariamente alcançada, uma vez que o algoritmo explora sistematicamente todas as ramificações válidas da árvore de busca sem saltar nenhum nó. A condição de parada, que consiste em retornar true quando não existem mais células com valor 0, garante que a solução só é declarada quando o tabuleiro está integralmente preenchido.

A correção do laço principal pode ser estabelecida por meio de uma invariante de laço: ao início de cada iteração sobre os valores candidatos de uma célula, o tabuleiro encontra-se em um estado parcialmente preenchido que é consistente com todas as restrições do Sudoku, ou seja, nenhuma das atribuições realizadas até aquele ponto viola unicidade de linha, coluna ou subgrade. Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada com pistas que não se contradizem entre si. É mantida a cada iteração, pois a atribuição de um novo valor à célula corrente só ocorre após a confirmação de SudokuValidator.isValid, que garante que o valor não cria nenhum conflito com o estado atual; caso nenhum valor satisfaça a verificação, a célula permanece inalterada e o algoritmo retrocede, restaurando o estado consistente anterior. É satisfeita na terminação, pois quando o laço encerra com retorno true, todas as células foram preenchidas e a invariante de consistência global do tabuleiro está integralmente preservada.

Além disso, toda atribuição realizada pelo algoritmo é precedida pela verificação SudokuValidator.isValid(tabuleiro, linha, coluna, numero), que confirma que o valor candidato não viola nenhuma das três restrições simultaneamente: unicidade na linha, unicidade na coluna e unicidade na subgrade 3×3. Como essa verificação ocorre antes de cada atribuição, qualquer estado parcial mantido durante a execução permanece consistente com as regras do jogo. Consequentemente, quando o algoritmo retorna true, o tabuleiro está completo e cada uma das 81 células satisfaz todas as restrições. Essa propriedade é formalmente validada pelos testes em BacktrackingSolverTest, que percorrem todas as células da solução retornada e invocam SudokuValidator.isValid célula a célula, após temporariamente zerá-las para evitar falso conflito da célula consigo mesma.

O algoritmo termina em tempo finito, uma vez que o número de células é estritamente limitado a 81, cada célula aceita no máximo nove valores distintos, e a árvore de busca gerada pelo mecanismo de retrocesso é finita e acíclica. A aciclicidade decorre do fato de que cada nível de recursão corresponde a uma célula vazia distinta: ao descer na recursão, o algoritmo sempre avança para uma célula ainda não preenchida, e ao retroceder, desfaz exatamente a última atribuição e tenta o próximo candidato na mesma célula antes de subir um nível. Dessa forma,

nenhum ramo da árvore é explorado mais de uma vez, e a profundidade máxima da pilha de recursão corresponde ao número de células vazias no tabuleiro inicial, que é sempre menor que 81, impedindo qualquer possibilidade de recursão infinita.

3.1.2. Análise de Desempenho

3.1.2.1. Espacial

A complexidade espacial do Backtracking é composta por dois componentes principais: o espaço ocupado pela representação do tabuleiro e o espaço consumido pela pilha de chamadas recursivas. O tabuleiro é representado pela classe SudokuBoard como uma matriz int[9][9], que ocupa espaço constante independentemente da instância resolvida, uma vez que as dimensões do Sudoku são fixas em 9×9. A matriz booleana auxiliar boolean[9][9], responsável por registrar as células fixas, acrescenta outro bloco de tamanho constante. O espaço total dedicado ao modelo de dados é, portanto, O(1).

O componente dominante na análise espacial é a pilha de recursão. Cada chamada recursiva a backtrack empilha um novo frame de ativação contendo as variáveis locais de controle dos laços: linha, coluna e número, além do endereço de retorno e da referência ao objeto Metrics. A profundidade máxima da pilha é igual ao número de células vazias no tabuleiro inicial, denotado por n, que varia de zero, no caso de um tabuleiro já resolvido, até aproximadamente 64 nas instâncias mais difíceis do Sudoku padrão. A complexidade espacial total é, portanto, O(n), com $n \leq 81$.

Um aspecto crítico para essa complexidade favorável é o fato de o algoritmo realizar todas as atribuições e desfazimentos in-place na mesma instância de SudokuBoard, sem criar cópias do tabuleiro a cada nível de recursão. Essa decisão de implementação é fundamental, pois se uma cópia fosse criada a cada chamada recursiva, o consumo de memória escalaria para $O(n \times 81)$, aumentando em cerca de duas ordens de grandeza o uso de memória nas instâncias mais difíceis.

3.1.2.2. Temporal

A análise da complexidade temporal do Backtracking para o Sudoku exige distinguir entre o pior caso teórico e o comportamento esperado na prática, considerando o papel fundamental das restrições na poda do espaço de busca. No pior caso teórico, o algoritmo explora todas as combinações possíveis de valores para as n células vazias antes de encontrar uma solução ou declarar insolubilidade. Com cada célula podendo receber até nove valores, o pior caso é $O(9^n)$. Para n = 81, isso resulta em $O(9^{81})$, um número da ordem de 10^{77}, computacionalmente impraticável. Na prática, contudo, esse limite superior é muito pessimista e nunca se materializa, pois as verificações de restrição eliminam a imensa maioria dos ramos logo nas primeiras atribuições, e a estrutura do jogo impõe uma densidade de restrições suficiente para tornar a busca tratável.

O mecanismo de poda reduz drasticamente o número de nós efetivamente explorados. Cada chamada a SudokuValidator.isValid tem custo O(1), pois percorre no máximo 27 células, nove da linha, nove da coluna e nove da subgrade 3×3 e a maioria dos valores candidatos é descartada antes mesmo de gerar uma chamada recursiva. O número real de nós visitados depende fortemente da instância: jogos com mais pistas impõem mais restrições e reduzem o espaço de busca de forma mais agressiva. Para as três instâncias utilizadas neste trabalho: fácil, médio e difícil, o número empírico de nós visitados é coletado pela classe Metrics e comparado diretamente com o Branch and Bound na análise experimental.

Cada nó visitado executa um laço de varredura linear sobre as 81 células para localizar a próxima célula vazia, com custo O(81) = O(1) para dimensões fixas. Em seguida, realiza até nove verificações de validade, cada uma com custo O(27) = O(1). O custo total por nó é, portanto, O(1), e o custo total da execução é diretamente proporcional ao número de nós visitados, sendo essa métrica a mais representativa do desempenho real do algoritmo e a que melhor captura a diferença entre instâncias de diferentes dificuldades.

O principal fator de ineficiência do Backtracking clássico reside na ausência de qualquer heurística para a seleção da próxima variável. Ao escolher sempre a primeira célula vazia em ordem de leitura, o algoritmo pode realizar um grande número de atribuições antes de alcançar uma célula altamente restrita que tornaria

o conflito evidente mais cedo na árvore de busca. Essa limitação motiva diretamente a estratégia adotada pelo Branch and Bound com heurística MRV, analisada na seção seguinte.

3.2. Branch and Bound

O Branch and Bound é uma técnica algorítmica de busca sistematizada que combina a exploração recursiva da árvore de decisão, característica do Backtracking, com mecanismos de seleção inteligente de variáveis e poda antecipada de ramos inviáveis. O nome da técnica reflete suas duas operações fundamentais: o branching, que consiste em ramificar o espaço de busca ao atribuir um valor a uma variável e gerar subproblemas menores, e o bounding, que consiste em avaliar, antes de explorar um ramo, se ele ainda tem condições de levar a uma solução válida, descartando-o precocemente caso contrário.

No contexto do Sudoku, a diferença central em relação ao Backtracking clássico reside na estratégia de seleção da próxima variável. Enquanto o Backtracking escolhe sempre a primeira célula vazia em ordem linear, o Branch and Bound utiliza a heurística MRV (Minimum Remaining Values), que seleciona a célula com o menor número de valores candidatos ainda válidos. A intuição por trás dessa heurística é direta: células mais restritas têm menos margem para erro e, portanto, devem ser preenchidas primeiro, pois um eventual conflito será detectado mais cedo na árvore, evitando a propagação de atribuições inconsistentes por muitos níveis de recursão antes do retrocesso.

A implementação está centralizada na classe BranchAndBoundSolver, que delega a seleção de variável à classe MRVHeuristic por meio da interface VariableOrderingHeuristic. Essa separação entre o algoritmo de busca e a estratégia de ordenação de variáveis segue o princípio de responsabilidade única e permite que diferentes heurísticas sejam plugadas ao solver sem alterar sua lógica interna.

A função de bound é implementada pelo método calcularBound, que é invocado após cada atribuição de valor a uma célula. O método percorre todas as células vazias restantes do tabuleiro e conta quantas delas ficaram sem nenhum candidato válido após a atribuição corrente. Se esse contador for maior que zero, o ramo é podado imediatamente: o valor atribuído é desfeito, um backtrack é registrado nas métricas e o algoritmo passa para o próximo candidato sem descer

na recursão. Apenas quando o bound retorna zero, indicando que nenhuma célula vazia ficou inviável, o algoritmo considera que o ramo é promissor e desce para o próximo nível. Essa verificação antecipada é o que distingue estruturalmente o Branch and Bound de um Backtracking com MRV: sem ela, os dois algoritmos seriam equivalentes em comportamento, diferindo apenas na ordem de seleção das células.

Uma diferença estrutural importante em relação ao Backtracking é a condição de parada. No Backtracking, a solução é detectada quando o laço de varredura não encontra mais nenhuma célula com valor 0. No Branch and Bound, a detecção é feita quando MRVHeuristic.selecionarCelula retorna null, indicando que não há mais células vazias. Essa distinção elimina a necessidade de varredura linear completa a cada nível de recursão para detectar o estado terminal.

Outra distinção relevante está na forma como os candidatos são obtidos. O método obterCandidatos do BranchAndBoundSolver constrói um array de até 10 posições, onde o índice 0 guarda a contagem de candidatos válidos e os índices 1 a 9 armazenam os valores candidatos, permitindo que sejam enumerados e testados diretamente, sem repetir a chamada a SudokuValidator.isValid para cada valor durante a iteração. Assim como no Backtracking, o modo de visualização passo a passo pode ser ativado via parâmetro booleano no construtor, sendo desativado nos experimentos em lote com new BranchAndBoundSolver(false).

3.2.1. Análise de Corretude

A corretude do Branch and Bound pode ser demonstrada pelas mesmas três propriedades examinadas para o Backtracking, sendo elas completude, validade e terminação, com atenção especial ao impacto da heurística MRV e da função de bound sobre cada uma delas.

O algoritmo é completo, ou seja, se existe uma solução para o jogo, ela será encontrada. A introdução da heurística MRV altera apenas a ordem em que os nós da árvore de busca são visitados, sem eliminar qualquer nó que devesse ser explorado. Toda célula vazia será eventualmente selecionada pela heurística, pois o critério de seleção é baseado exclusivamente na contagem de candidatos válidos no estado atual do tabuleiro, não havendo viés que cause a omissão permanente de

qualquer célula. A prova de completude é análoga à do Backtracking: o espaço de busca é finito, a varredura é exaustiva dentro de cada ramificação, e o retrocesso garante que todas as alternativas sejam exploradas antes de declarar insolubilidade. A função de bound não compromete a completude, pois ela poda exclusivamente ramos em que alguma célula vazia ficou sem candidatos válidos, situação que, por definição, não pode levar a nenhuma solução válida, de modo que nenhum ramo promissor é descartado.

A correção do laço de iteração sobre os candidatos pode ser estabelecida por meio de uma invariante de laço: ao início de cada iteração, o tabuleiro encontra-se em um estado parcialmente preenchido que é consistente com todas as restrições do Sudoku, de modo que nenhuma das atribuições realizadas até aquele ponto viola unicidade de linha, coluna ou subgrade. Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada cujas pistas não se contradizem entre si. É mantida a cada iteração, pois apenas candidatos previamente validados pelo método obterCandidatos são atribuídos ao tabuleiro: o método invoca SudokuValidator.isValid para cada valor de 1 a 9 e inclui na lista somente os valores que não violam nenhuma das três restrições, de forma que qualquer valor efetivamente atribuído já está garantidamente livre de conflitos com o estado corrente. Caso a lista de candidatos esteja vazia, nenhuma atribuição é realizada, o algoritmo retrocede e o estado consistente anterior é restaurado. É satisfeita na terminação, pois quando o laço encerra com retorno true, todas as células foram preenchidas e a invariante de consistência global está integralmente preservada.

Como apenas candidatos previamente validados são atribuídos ao tabuleiro, qualquer estado parcial mantido durante a execução permanece sempre consistente com as regras do jogo. Quando o algoritmo retorna true, o tabuleiro está completo e todas as restrições são satisfeitas, sendo essa propriedade verificada formalmente pelos testes em BranchAndBoundSolverTest e SolverComparisonTest.

O algoritmo termina em tempo finito pelos mesmos argumentos do Backtracking. O número de células é limitado a 81, cada célula aceita no máximo nove valores e a árvore de busca gerada pelo mecanismo de retrocesso é finita e

acíclica. A profundidade máxima da pilha de recursão continua sendo O(n), onde n é o número de células vazias no tabuleiro inicial.

A heurística MRV e a função de bound atuam em conjunto como os dois principais mecanismos de eficiência do Branch and Bound em relação ao Backtracking clássico. A heurística MRV contribui ao selecionar a célula com menos candidatos, detectando situações de falha, células com zero candidatos válidos, mais cedo na árvore de busca: quando isso ocorre, a heurística seleciona essa célula com prioridade máxima e o algoritmo retorna false imediatamente, sem explorar nenhuma atribuição para aquela posição. A função de bound, implementada pelo método calcularBound, atua de forma complementar: após cada atribuição, ela percorre todas as células vazias restantes e verifica se alguma ficou sem candidatos válidos; caso positivo, o ramo é podado antes mesmo de descer na recursão, desfazendo a atribuição corrente sem precisar explorar os subproblemas gerados por ela. Sem essa função de bound, o algoritmo se reduziria a um Backtracking com MRV, diferindo do Branch and Bound apenas na nomenclatura. É a combinação das duas estratégias que caracteriza a técnica e explica seu ganho de eficiência sobre o Backtracking simples.

3.2.2. Análise de Desempenho

3.2.2.1. Espacial

A complexidade espacial do Branch and Bound compartilha a mesma estrutura de análise do Backtracking, com um componente adicional introduzido pela heurística MRV.

O espaço ocupado pelo modelo de dados é idêntico ao do Backtracking: a matriz int[9][9] de SudokuBoard e a matriz booleana auxiliar boolean[9][9] ocupam espaço constante O(1). O algoritmo também opera in-place, sem criar cópias do tabuleiro a cada nível de recursão, mantendo o mesmo perfil favorável de uso de memória.

A pilha de recursão continua sendo o componente dominante, com profundidade máxima igual ao número de células vazias *n* e complexidade O(n).

Cada frame de ativação armazena a referência à célula selecionada pela heurística, o array de candidatos e as variáveis de controle do laço de iteração sobre os candidatos.

O componente adicional introduzido pelo Branch and Bound é o array de candidatos gerado a cada chamada recursiva por obterCandidatos: um array int[10] alocado localmente, com tamanho fixo independente da instância. Como esse array é local ao frame de cada chamada recursiva, seu espaço é liberado pelo garbage collector ao término de cada chamada, e o espaço total ocupado simultaneamente é proporcional à profundidade da pilha: $O(n \times 10) = O(n)$. Da mesma forma, a heurística MRV realiza uma varredura interna sobre todas as células vazias a cada chamada, mas não aloca estruturas de dados persistentes, apenas variáveis de controle locais, sem impacto adicional na complexidade espacial assintótica.

A complexidade espacial total do Branch and Bound é, portanto, $O(n)$, idêntica à do Backtracking, com $n \leq 81$ para qualquer instância de Sudoku 9×9.

3.2.2.2. Temporal

A análise temporal do Branch and Bound envolve dois fatores que não estão presentes no Backtracking: o custo adicional da heurística MRV e da função de bound a cada nível de recursão, e a redução no número total de nós visitados proporcionada pela seleção inteligente de variáveis e pela poda antecipada. O limite superior teórico do Branch and Bound continua sendo $O(9^n)$, o mesmo do Backtracking, pois no pior caso absoluto a heurística MRV e a função de bound não conseguem reduzir o espaço de busca de forma assintoticamente significativa. Na prática, contudo, esse limite é ainda menos atingível do que no Backtracking, uma vez que a detecção precoce de conflitos elimina ramos muito antes de serem explorados em profundidade.

A cada chamada recursiva, antes de qualquer atribuição, o algoritmo invoca MRVHeuristic.selecionarCelula, que percorre todas as células vazias do tabuleiro. Para cada célula vazia encontrada, o método obterCandidatos verifica os nove valores possíveis contra as restrições, com custo $O(1)$ por valor. No pior caso, com

n células vazias e nove verificações por célula, o custo da heurística por nó é O(n × 9) = O(n). Após cada atribuição, o método calcularBound percorre novamente todas as células vazias restantes e conta os candidatos válidos de cada uma, com o mesmo custo O(n × 9) = O(n). Os dois componentes somados resultam em um overhead de O(n) por nó visitado, constante na ordem assintótica mas com uma constante multiplicativa maior do que a da heurística isolada. Comparado ao Backtracking, que tem custo O(1) por nó para a seleção de variável e não executa nenhuma função de bound, o Branch and Bound paga um custo adicional de O(n) por nó visitado. Isso significa que, para instâncias simples, onde o Backtracking já visita poucos nós, o Branch and Bound pode ser marginalmente mais lento devido a esse overhead.

A vantagem do Branch and Bound se manifesta quando o número de nós que a heurística MRV e a função de bound evitam explorar é suficientemente grande para compensar o custo adicional por nó. Ao selecionar sempre a célula mais restrita, o algoritmo concentra os retrocessos nas posições de maior conflito, propagando as restrições de forma mais eficiente pela árvore de busca. A função de bound complementa esse efeito ao podar ramos inteiros antes mesmo de descer na recursão, eliminando subproblemas que o Backtracking só descartaria após explorá-los parcialmente. Em instâncias de dificuldade média e alta, onde o espaço de busca é mais vasto, a redução no número de nós visitados tende a ser expressiva, resultando em tempo total de execução significativamente inferior ao do Backtracking, mesmo com o custo maior por nó.

Seja V_bt o número de nós visitados pelo Backtracking e V_bb o número de nós visitados pelo Branch and Bound para uma mesma instância. O custo total do Backtracking é O(V_bt) e o do Branch and Bound é O(V_bb × n). O Branch and Bound é mais eficiente quando V_bb × n é muito menor que V_bt, o que ocorre nas instâncias onde a heurística MRV e a função de bound reduzem drasticamente a exploração. Para instâncias fáceis, onde ambos os algoritmos visitam poucos nós, a diferença é pequena e o overhead combinado da heurística e do bound pode até tornar o Branch and Bound ligeiramente mais lento. Para instâncias difíceis, a redução em V_bb domina e o Branch and Bound apresenta desempenho superior, o que é quantificado empiricamente na seção de resultados por meio das métricas coletadas pela classe Metrics.

3.3. Estratégia Gulosa (Greedy)

A Estratégia Gulosa é uma técnica algorítmica de busca orientada por escolhas locais informadas, que combina a exploração recursiva da árvore de decisão com heurísticas de seleção de variável e de ordenação de valores para maximizar a flexibilidade da busca a cada passo. O nome da técnica remete à sua propriedade central: a cada nível da recursão, o algoritmo toma a decisão que parece localmente ótima segundo dois critérios complementares, sem recalcular globalmente o impacto de cada escolha, confiando que boas decisões locais encadeadas produzirão um bom caminho global.

No contexto do Sudoku, a diferença central em relação ao Backtracking clássico reside na combinação de duas heurísticas aplicadas em estágios distintos da expansão de cada nó. O Backtracking seleciona sempre a primeira célula vazia em ordem linear e testa os valores de 1 a 9 em sequência arbitrária. A Estratégia Gulosa, por sua vez, seleciona a célula com o menor número de candidatos válidos pela heurística MRV e, para essa célula, ordena os candidatos pelo critério LCV (Least Constraining Value), priorizando os valores que menos restringem os domínios das células vizinhas. A intuição por trás dessas escolhas é direta: atacar primeiro as células mais restritas aumenta a chance de detectar contradições nos níveis altos da árvore, e preferir os valores menos restritivos preserva mais flexibilidade para as atribuições futuras, reduzindo a probabilidade de becos sem saída prematuros.

A implementação está centralizada na classe GreedySolver, que delega a seleção de variável à classe MRVHeuristic por meio da interface VariableOrderingHeuristic e a ordenação de candidatos ao método ordenarPorLCV, responsável por calcular o custo de restrição de cada valor candidato. Essa separação entre o algoritmo de busca, a heurística de seleção de variável e o critério de ordenação de valores segue o princípio de responsabilidade única e permite que diferentes combinações de heurísticas sejam plugadas ao solver sem alterar sua lógica interna.

O custo de restrição de um valor candidato é calculado pelo método calcularCustoLCV, que é invocado para cada candidato antes da ordenação. O

método percorre as células vizinhas da célula selecionada, isto é, as células que compartilham linha, coluna ou subgrade, e conta quantas delas teriam aquele valor eliminado de seu domínio caso ele fosse atribuído. Quanto menor esse contador, mais o valor preserva os domínios dos vizinhos e, portanto, maior é sua prioridade na ordenação. Após a ordenação, os candidatos são tentados na sequência resultante: para cada candidato, o valor é atribuído, a chamada recursiva é realizada e, se ela retornar falso, a atribuição é desfeita e o próximo candidato é tentado. O retrocesso é registrado nas métricas apenas quando nenhum candidato conduz à solução, indicando que aquele estado não possui extensão válida.

Uma diferença estrutural importante em relação ao Backtracking está na condição de parada. No Backtracking, a solução é detectada quando a varredura linear não encontra mais nenhuma célula com valor 0. Na Estratégia Gulosa, assim como no Branch and Bound, a detecção é feita quando MRVHeuristic.selecionarCelula retorna null, indicando que não há mais células vazias. Essa distinção elimina a necessidade de varredura completa a cada nível de recursão para detectar o estado terminal.

Outra distinção relevante está na forma como os candidatos são obtidos e processados. O método obterCandidatos do GreedySolver coleta os valores válidos para a célula selecionada por MRV, invocando SudokuValidator.isValid para cada valor de 1 a 9, e os repassa ao método ordenarPorLCV, que os reorganiza por custo crescente de restrição antes da iteração. Essa separação entre coleta e ordenação de candidatos contrasta com o Branch and Bound, que coleta candidatos e aplica apenas a função de bound antes da iteração, sem reordenar os valores. Assim como nos demais solvers, o modo de visualização passo a passo pode ser ativado via parâmetro booleano no construtor, sendo desativado nos experimentos em lote com new GreedySolver(false).

3.3.1. Análise de Corretude

A corretude da Estratégia Gulosa pode ser demonstrada pelas mesmas três propriedades examinadas para o Backtracking e o Branch and Bound, sendo elas completude, validade e terminação, com atenção especial ao impacto das heurísticas MRV e LCV sobre cada uma delas.

O algoritmo é completo, ou seja, se existe uma solução para o jogo, ela será encontrada. A introdução das heurísticas MRV e LCV altera apenas a ordem em que os nós da árvore de busca são visitados, sem eliminar qualquer nó que devesse ser explorado. A heurística MRV seleciona variáveis com base na contagem de candidatos válidos no estado atual do tabuleiro, sem omitir permanentemente nenhuma célula. A ordenação LCV reorganiza os candidatos de uma célula por custo de restrição, mas mantém todos os candidatos válidos na lista: nenhum valor é descartado pela ordenação, apenas reposicionado. A prova de completude é análoga à do Backtracking, o espaço de busca é finito, a varredura é exaustiva dentro de cada ramificação e o retrocesso garante que todas as alternativas sejam exploradas antes de declarar insolubilidade. As heurísticas não comprometem a completude, pois não removem ramos da árvore: apenas determinam a sequência em que eles são visitados.

A validade é estabelecida por uma invariante recursiva: em toda chamada, o tabuleiro parcial respeita as regras de unicidade em linha, coluna e subgrade. Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada cujas pistas não se contradizem entre si. É mantida a cada iteração, pois apenas candidatos previamente validados por SudokuValidator.isValid são incluídos na lista ordenada pelo LCV: o método verifica cada valor de 1 a 9 contra as três restrições do Sudoku e inclui na lista somente os valores livres de conflito com o estado corrente, de forma que qualquer valor efetivamente atribuído já está garantidamente livre de conflitos. Caso a lista de candidatos esteja vazia, nenhuma atribuição é realizada, o algoritmo retrocede e o estado consistente anterior é restaurado. É satisfeita na terminação, pois quando o algoritmo retorna true, todas as células foram preenchidas e a invariante de consistência global está integralmente preservada.

Como apenas candidatos previamente validados são atribuídos ao tabuleiro, qualquer estado parcial mantido durante a execução permanece sempre consistente com as regras do jogo. A ordenação LCV não interfere nessa invariante: ela atua exclusivamente sobre a sequência de tentativas dentro de um mesmo nó, sem alterar o critério de inclusão dos candidatos na lista. Quando o algoritmo retorna true, o tabuleiro está completo e todas as restrições são satisfeitas, sendo essa propriedade verificada formalmente pelos testes em GreedySolverTest e SolverComparisonTest.

O algoritmo termina em tempo finito pelos mesmos argumentos do Backtracking. O número de células é limitado a 81, cada célula aceita no máximo nove valores e a árvore de busca gerada pelo mecanismo de retrocesso é finita e acíclica. A profundidade máxima da pilha de recursão continua sendo O(n), onde n é o número de células vazias no tabuleiro inicial. As heurísticas MRV e LCV atuam como os dois principais mecanismos de eficiência da Estratégia Gulosa em relação ao Backtracking clássico: a MRV concentra os retrocessos nas posições de maior conflito, detectando falhas mais cedo na árvore, enquanto o LCV reduz a frequência com que atribuições correntes inviabilizam células vizinhas, diminuindo o número de retrocessos necessários para alcançar a solução.

3.3.2. Análise de Desempenho

3.3.2.1. Espacial

A complexidade espacial da Estratégia Gulosa compartilha a estrutura de análise do Backtracking e do Branch and Bound, com um componente adicional introduzido pela ordenação LCV.

O espaço ocupado pelo modelo de dados é idêntico ao dos demais algoritmos: a matriz int[9][9] de SudokuBoard e a matriz booleana auxiliar boolean[9][9] ocupam espaço constante O(1). O algoritmo também opera in-place,

sem criar cópias do tabuleiro a cada nível de recursão, mantendo o mesmo perfil favorável de uso de memória.

A pilha de recursão continua sendo o componente dominante, com profundidade máxima igual ao número de células vazias n e complexidade O(n). Cada frame de ativação armazena a referência à célula selecionada pela heurística MRV, o array de candidatos ordenados pelo LCV e as variáveis de controle do laço de iteração sobre os candidatos.

O componente adicional introduzido pela Estratégia Gulosa é o array de custos gerado a cada chamada recursiva por calcularCustoLCV: um array de até nove entradas, alocado localmente para armazenar o custo de restrição de cada candidato antes da ordenação. Como esse array é local ao frame de cada chamada recursiva, seu espaço é liberado ao término de cada chamada, e o espaço total ocupado simultaneamente é proporcional à profundidade da pilha: $O(n \times 9) = O(n)$. Da mesma forma, a heurística MRV realiza uma varredura interna sobre todas as células vazias a cada chamada, mas não aloca estruturas de dados persistentes, apenas variáveis de controle locais, sem impacto adicional na complexidade espacial assintótica.

A complexidade espacial total da Estratégia Gulosa é, portanto, O(n), idêntica à do Backtracking e do Branch and Bound, com $n \leq 81$ para qualquer instância de Sudoku 9×9.

3.3.2.2. Temporal

A análise temporal da Estratégia Gulosa envolve dois fatores que não estão presentes no Backtracking: o custo adicional das heurísticas MRV e LCV a cada nível de recursão, e a redução no número total de nós visitados proporcionada pela seleção inteligente de variável e pela ordenação de valores. O limite superior teórico continua sendo $O(9^n)$, o mesmo do Backtracking e do Branch and Bound, pois no pior caso absoluto as heurísticas não conseguem reduzir o espaço de busca de forma assintoticamente significativa. Na prática, contudo, esse limite é ainda menos atingível do que no Backtracking, uma vez que a detecção precoce de conflitos pela

MRV e a preservação de domínios pelo LCV eliminam ramos antes de serem explorados em profundidade.

A cada chamada recursiva, o algoritmo invoca MRVHeuristic.selecionarCelula, que percorre todas as células vazias do tabuleiro com custo O(n), e em seguida invoca calcularCustoLCV para cada candidato da célula selecionada. O cálculo do custo de um candidato percorre as células vizinhas da célula selecionada, no máximo 20 células distintas entre linha, coluna e subgrade, desconsideradas sobreposições, e verifica a validade do candidato em cada uma delas, com custo O(1) por verificação. Como o número de candidatos é no máximo 9 e o número de vizinhos é constante para o Sudoku 9×9, o custo total de calcularCustoLCV por nó é O(9 × 20) = O(1) para dimensão fixa. Somando os dois componentes, o overhead por nó visitado é O(n), dominado pela heurística MRV, com uma constante multiplicativa maior do que a do Branch and Bound, que não executa a ordenação LCV.

Comparado ao Branch and Bound, que tem custo O(n) por nó devido à heurística MRV e à função calcularBound, a Estratégia Gulosa substitui o custo do bound prospectivo pelo custo da ordenação LCV, ambos O(1) para dimensão fixa. A diferença entre as duas técnicas não está, portanto, no custo por nó, mas no mecanismo de redução de busca: o Branch and Bound poda ramos inteiros antes de descer na recursão, eliminando subproblemas por completo; a Estratégia Gulosa não poda ramos, mas ordena os candidatos de forma a tentar primeiro os valores com maior probabilidade de sucesso, reduzindo o número de retrocessos necessários para alcançar a solução.

Seja V_{bt} o número de nós visitados pelo Backtracking, V_{bb} o número de nós visitados pelo Branch and Bound e V_g o número de nós visitados pela Estratégia Gulosa para uma mesma instância. O custo total do Backtracking é $O(V_{bt})$, o do Branch and Bound é $O(V_{bb} \times n)$ e o da Estratégia Gulosa é $O(V_g \times n)$. A Estratégia Gulosa é mais eficiente que o Backtracking quando V_g é suficientemente menor que V_{bt} para compensar o overhead por nó. A comparação com o Branch and Bound depende da instância: em problemas onde a poda prospectiva do bound é altamente eficaz, o Branch and Bound tende a visitar menos nós; em instâncias onde a ordenação LCV orienta bem a busca desde os primeiros

níveis da árvore, a Estratégia Gulosa pode competir ou superar o Branch and Bound ao evitar retrocessos custosos por meio de boas escolhas iniciais. Essa distinção é quantificada empiricamente na seção de resultados por meio das métricas coletadas pela classe Metrics.

3.4. Programação Dinâmica

A Programação Dinâmica é uma técnica algorítmica que resolve problemas complexos decompondo-os em subproblemas menores e armazenando os resultados desses subproblemas para evitar recomputação redundante. No contexto do Sudoku, ela é aplicada na forma de memoização combinada com busca em profundidade e poda por restrições. O nome da técnica remete à sua propriedade central: subproblemas que se repetem ao longo da exploração do espaço de busca são resolvidos uma única vez, e seus resultados são registrados em uma estrutura de memória para consulta imediata nas ocorrências seguintes.

No contexto do Sudoku, a diferença central em relação ao Backtracking clássico reside no tratamento de estados parciais já explorados. Enquanto o Backtracking pode revisitar configurações intermediárias do tabuleiro por caminhos distintos na árvore de busca, a Programação Dinâmica serializa cada estado parcial em uma chave canônica e o registra como estado morto caso a exploração daquele estado não tenha levado a nenhuma solução. Nas ocorrências subsequentes do mesmo estado, o algoritmo reconhece imediatamente a inviabilidade e retrocede sem repetir a expansão. A intuição por trás dessa estratégia é direta: se um estado parcial já foi provado insolúvel, nenhum caminho que passe por ele poderá levar a uma solução válida, independentemente da sequência de atribuições que o gerou.

A implementação está centralizada na classe DynamicProgrammingSolver, que mantém um conjunto de estados mortos compartilhados entre todas as chamadas recursivas. A seleção da próxima variável é delegada à heurística MRV, por meio da interface VariableOrderingHeuristic, de modo análogo ao Branch and Bound. Essa separação entre a lógica de busca, o mecanismo de memoização e a estratégia de ordenação de variáveis segue o princípio de responsabilidade única e

permite que diferentes heurísticas sejam plugadas ao solver sem alterar sua lógica central.

A função de memoização é implementada pela serialização do tabuleiro em uma chave de 81 caracteres, invocada no início de cada chamada recursiva. O método percorre todas as 81 células do tabuleiro, concatena seus valores em uma string e consulta o conjunto de estados mortos. Se a chave já estiver presente, o ramo é podado imediatamente: o algoritmo retorna falha sem executar nenhuma atribuição, sem invocar a heurística e sem descer na recursão. Apenas quando a chave é nova o algoritmo prossegue: seleciona a célula com menor número de candidatos válidos pela heurística MRV, tenta os valores de 1 a 9, valida cada tentativa com SudokuValidator.isValid, avança recursivamente quando a atribuição é consistente e desfaz a jogada quando necessário. Se nenhum valor funcionar para aquele estado, a chave é inserida no conjunto de estados mortos antes do retrocesso, garantindo que futuras ocorrências daquele estado sejam descartadas em tempo constante.

Uma diferença estrutural relevante em relação ao Backtracking está na condição de parada. No Backtracking, a solução é detectada quando a varredura linear não encontra mais nenhuma célula com valor 0. Na Programação Dinâmica, assim como no Branch and Bound, a detecção é feita quando MRVHeuristic.selecionarCelula retorna null, indicando que não há mais células vazias. Essa distinção elimina a necessidade de varredura completa a cada nível de recursão para detectar o estado terminal.

Outra distinção relevante está na natureza da poda. No Branch and Bound, a função calcularBound avalia prospectivamente se o ramo atual ainda tem condições de levar a uma solução. Na Programação Dinâmica, a poda é retrospectiva: o algoritmo não antecipa o futuro do ramo corrente, mas reconhece estados já provados insolúveis e os descarta com base na memória acumulada de execuções anteriores. As duas estratégias são complementares e atuam em dimensões distintas da árvore de busca. Assim como no Branch and Bound, o modo de visualização passo a passo pode ser ativado via parâmetro booleano no construtor, sendo desativado nos experimentos em lote com new DynamicProgrammingSolver(false).

3.4.1. Análise de Corretude

A corretude da Programação Dinâmica pode ser demonstrada pelas mesmas três propriedades examinadas para o Backtracking e o Branch and Bound, sendo elas completude, validade e terminação, com atenção especial ao impacto da memoização sobre cada uma delas.

O algoritmo é completo, ou seja, se existe uma solução para o jogo, ela será encontrada. A memoização registra exclusivamente estados para os quais já existe prova de insolubilidade: um estado só é inserido no conjunto de estados mortos após o esgotamento de todos os seus candidatos válidos sem que nenhum deles tenha levado a uma solução. Portanto, nenhum estado promissor é descartado: estados com potencial de solução nunca são inseridos no conjunto e sempre são explorados integralmente. A prova de completude é análoga à do Backtracking, o espaço de busca é finito, a varredura é exaustiva dentro de cada ramificação e o retrocesso garante que todas as alternativas sejam exploradas antes de declarar insolubilidade, com o acréscimo de que a memoização apenas evita recomputação de estados já esgotados, sem introduzir viés de omissão.

A validade é estabelecida por uma invariante recursiva: em toda chamada, o tabuleiro parcial respeita as regras de unicidade em linha, coluna e subgrade. Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada cujas pistas não se contradizem entre si. É mantida a cada iteração, pois apenas valores previamente validados por SudokuValidator.isValid são atribuídos ao tabuleiro: o método verifica cada valor de 1 a 9 contra as três restrições do Sudoku e autoriza a atribuição somente dos valores livres de conflito com o estado corrente. Caso nenhum valor seja válido, nenhuma atribuição é realizada, o estado é memoizado como morto, o algoritmo retrocede e o estado consistente anterior é restaurado. É satisfeita na terminação, pois quando o algoritmo retorna true, todas as células foram preenchidas e a invariante de consistência global está integralmente preservada.

Como apenas candidatos previamente validados são atribuídos ao tabuleiro, qualquer estado parcial mantido durante a execução permanece sempre consistente com as regras do jogo. A memoização não interfere nessa invariante: ela atua apenas no controle de fluxo da busca, descartando estados inviáveis na entrada da

chamada, antes de qualquer tentativa de atribuição. Quando o algoritmo retorna true, o tabuleiro está completo e todas as restrições são satisfeitas, sendo essa propriedade verificada formalmente pelos testes em DynamicProgrammingSolverTest e SolverComparisonTest.

O algoritmo termina em tempo finito pelos mesmos argumentos do Backtracking. O número de células é limitado a 81, cada célula aceita no máximo nove valores e a árvore de busca gerada pelo mecanismo de retrocesso é finita e acíclica. A profundidade máxima da pilha de recursão continua sendo O(n), onde n é o número de células vazias no tabuleiro inicial. Adicionalmente, a memoização reforça a terminação ao impedir que o algoritmo reexpanda estados já visitados: cada estado é expandido no máximo uma vez, após o que é ou resolvido com sucesso ou inserido no conjunto de estados mortos e descartado em todas as ocorrências futuras.

3.4.2. Análise de Desempenho

3.4.2.1. Espacial

A complexidade espacial da Programação Dinâmica compartilha a estrutura de análise do Backtracking e do Branch and Bound, com um componente adicional introduzido pela estrutura de memoização.

O espaço ocupado pelo modelo de dados é idêntico ao dos demais algoritmos: a matriz int[9][9] de SudokuBoard e a matriz booleana auxiliar boolean[9][9] ocupam espaço constante O(1). O algoritmo também opera in-place, sem criar cópias do tabuleiro a cada nível de recursão, mantendo o mesmo perfil favorável de uso de memória.

A pilha de recursão continua sendo um dos componentes dominantes, com profundidade máxima igual ao número de células vazias n e complexidade O(n). Cada frame de ativação armazena a chave serializada do estado corrente, a referência à célula selecionada pela heurística MRV e as variáveis de controle do laço de iteração sobre os candidatos.

O componente adicional e diferencial da Programação Dinâmica é o conjunto de estados mortos: uma estrutura do tipo HashSet<String> que armazena as chaves de 81 caracteres dos estados já provados insolúveis. O número de entradas nesse

conjunto depende da quantidade de estados inviáveis efetivamente visitados durante a execução, denotada por M. Cada entrada ocupa espaço proporcional ao tamanho da chave, que é fixo em 81 caracteres para o Sudoku 9×9. Essa escolha introduz um custo de memória adicional O(M) em troca da eliminação de recomputações, padrão característico de Programação Dinâmica top-down com memoização.

A complexidade espacial total da Programação Dinâmica é, portanto, O(n + M), onde n ≤ 81 é o número de células vazias e M é o número de estados mortos acumulados. Em instâncias onde a busca visita muitos estados inviáveis distintos, M pode crescer significativamente, tornando a memória o fator limitante em comparação com o Branch and Bound, cuja complexidade espacial é O(n).

3.4.2.2. Temporal

A análise temporal da Programação Dinâmica envolve dois fatores que não estão presentes no Backtracking simples: o custo adicional da serialização do estado e da consulta ao conjunto de estados mortos a cada nível de recursão, e a redução no número total de nós visitados proporcionada pelo reconhecimento imediato de estados inviáveis já explorados.

O limite superior teórico continua sendo $O(9^n)$, o mesmo do Backtracking e do Branch and Bound, pois no pior caso absoluto todos os estados visitados são distintos e a memoização não consegue eliminar nenhuma reexpansão. Na prática, contudo, esse limite é menos atingível quando a estrutura do problema gera estados parciais repetidos por caminhos distintos na árvore de busca, situação em que a memoização elimina recomputações inteiras.

A cada chamada recursiva, antes de qualquer atribuição, o algoritmo serializa o tabuleiro em uma string de 81 caracteres e consulta o conjunto de estados mortos. A serialização percorre as 81 células com custo O(1) por célula, totalizando O(81) = O(1) para o Sudoku de dimensão fixa. A consulta ao HashSet tem custo esperado O(1) por operação. Em seguida, caso o estado seja novo, o algoritmo invoca MRVHeuristic.selecionarCelula, com custo O(n) análogo ao do Branch and Bound. O overhead total por nó visitado é, portanto, O(n), dominado pela heurística MRV, com uma constante aditiva associada à serialização.

Comparado ao Branch and Bound, que também tem custo O(n) por nó devido à heurística e à função calcularBound, a Programação Dinâmica substitui o custo do bound prospectivo pelo custo da serialização e da inserção no conjunto de estados mortos, ambos O(1) para dimensão fixa. A diferença entre as duas técnicas não está, portanto, no custo por nó, mas no critério de poda: o Branch and Bound poda ramos com base em uma avaliação prospectiva do estado futuro, enquanto a Programação Dinâmica poda com base no reconhecimento retrospectivo de estados já explorados.

Seja V_bt o número de nós visitados pelo Backtracking, V_bb o número de nós visitados pelo Branch and Bound e V_dp o número de nós visitados pela Programação Dinâmica para uma mesma instância. O custo total do Backtracking é O(V_bt), o do Branch and Bound é O(V_bb × n) e o da Programação Dinâmica é O(V_dp × n). A Programação Dinâmica é mais eficiente que o Backtracking quando V_dp é suficientemente menor que V_bt para compensar o overhead por nó. A comparação com o Branch and Bound depende da natureza da instância: em problemas onde estados parciais idênticos são gerados por múltiplos caminhos, a memoização reduz V_dp de forma expressiva; em instâncias onde a árvore de busca não apresenta sobreposição de subproblemas, a vantagem da memoização é nula e o Branch and Bound tende a ser superior pela eficácia de sua poda prospectiva. Essa distinção é quantificada empiricamente na seção de resultados por meio das métricas coletadas pela classe Metrics.

4. EXPERIMENTOS E RESULTADOS

4.1. Instâncias de Testes

Os experimentos foram conduzidos sobre três instâncias de Sudoku 9×9, classificadas nos níveis fácil, médio e difícil, cada uma armazenada em um arquivo de texto no formato de matriz 9×9 com valores separados por espaço, onde o dígito 0 representa células vazias. As instâncias foram selecionadas de forma a cobrir um espectro de dificuldade crescente, variando o número de células preenchidas inicialmente e a distribuição das pistas pelo tabuleiro.

A instância fácil apresenta maior número de células preenchidas, reduzindo o espaço de busca e permitindo que todos os algoritmos alcancem a solução com poucos nós visitados. A instância média possui um número intermediário de pistas, exigindo maior exploração da árvore de decisão. A instância difícil é a mais esparsa, com menos células preenchidas inicialmente, ampliando o espaço de possibilidades e tornando a busca mais suscetível a retrocessos.

Cada combinação de algoritmo e nível de dificuldade foi executada dez vezes consecutivas pelo PerformanceComparisonTest, com recarregamento do tabuleiro a partir do arquivo e instanciação de um novo objeto Metrics a cada execução, garantindo estado inicial idêntico entre as repetições e isolamento dos contadores de desempenho.

4.2. Resultados Obtidos

Os resultados apresentados correspondem às médias e desvios padrão calculados sobre as dez execuções de cada combinação de algoritmo e nível de dificuldade.

Algoritmos	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados	Backtracks
Backtracking	0,316	0,672	70	50
Branch and Bound	0,272	0,231	20	0
Greedy	0,159	0,049	20	0
Dynamic Programming	0,245	0,159	20	0

Tabela 3 - Resultados para o Sudoku fácil.

Algoritmos	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados	Backtracks
Backtracking	0,060	0,047	86	57
Branch and Bound	0,224	0,050	29	0
Greedy	0,197	0,114	29	0
Dynamic Programming	0,117	0,025	29	0

Tabela 4 - Resultados para o Sudoku médio.

Algoritmos	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados	Backtracks
Backtracking	0,053	0,008	85	46
Branch and Bound	0,395	0,033	52	14
Greedy	0,219	0,043	42	3
Dynamic Programming	0,446	0,565	53	14

Tabela 5 - Resultados para o Sudoku difícil.

Na instância fácil, os algoritmos com heurística MRV, Branch and Bound, Greedy e Programação Dinâmica, visitaram apenas 20 nós cada, contra 70 do Backtracking, e não registraram nenhum backtrack, enquanto o Backtracking executou 50 retrocessos. Apesar dessa diferença expressiva em nós visitados, o Greedy foi o único algoritmo a apresentar tempo médio notavelmente inferior (0,159

ms), enquanto Branch and Bound e Programação Dinâmica ficaram próximos ao Backtracking em tempo absoluto, evidenciando o overhead da heurística por nó.

Na instância média, o padrão se repete: os três algoritmos com MRV visitaram 29 nós contra 86 do Backtracking, sem backtracks. O Backtracking, no entanto, registrou o menor tempo médio de todos (0,060 ms), resultado que ilustra de forma clara o tradeoff entre custo por nó e número de nós visitados: o custo adicional das heurísticas superou o ganho obtido com a redução da exploração nessa instância específica.

Na instância difícil, o Backtracking continuou sendo o mais rápido em tempo absoluto (0,053 ms), embora tenha visitado 85 nós e realizado 46 backtracks. O Greedy apresentou o menor número de nós visitados (42) e de backtracks (3) entre os algoritmos com heurística, com tempo médio de 0,219 ms. Branch and Bound e Programação Dinâmica visitaram respectivamente 52 e 53 nós, com 14 backtracks cada, e foram os mais lentos em tempo absoluto, com médias de 0,395 ms e 0,446 ms. O desvio padrão elevado da Programação Dinâmica nessa instância (0,565 ms) indica maior variabilidade entre execuções, possivelmente relacionada ao custo de manutenção e consulta ao conjunto de estados mortos em instâncias com maior profundidade de busca.

Os resultados obtidos revelam que, nas três instâncias testadas, o Backtracking apresentou o menor tempo de execução absoluto apesar de visitar significativamente mais nós e realizar mais retrocessos. Esse comportamento aparentemente contraditório é explicado pelo overhead computacional que os algoritmos com heurística pagam por nó visitado: a varredura completa das células vazias pelo MRV, a ordenação de candidatos pelo LCV no Greedy, a função de bound no Branch and Bound e a serialização e consulta ao conjunto de estados mortos na Programação Dinâmica introduzem custos fixos por nó que, nas instâncias utilizadas, superaram o benefício da redução no número de nós explorados.

Esse resultado sugere que as instâncias testadas não possuem espaço de busca suficientemente vasto para evidenciar a vantagem assintótica dos algoritmos mais sofisticados. Em instâncias com maior número de células vazias e distribuição

de pistas que exija exploração mais profunda da árvore de decisão, espera-se que a redução no número de nós visitados proporcionada pelas heurísticas passe a dominar o overhead por nó, invertendo a relação de desempenho observada aqui.

5. ANÁLISE E DISCUSSÃO

5.1. Comparação das Estratégias

5.1.1. Visão Geral das Técnicas

Os quatro algoritmos implementados compartilham a mesma estrutura fundamental de busca recursiva com retrocesso, diferindo na estratégia de seleção de variável, na ordenação de valores candidatos e nos mecanismos adicionais de poda ou memoização. Essa base comum permite uma comparação controlada entre as técnicas, isolando o impacto de cada decisão de projeto sobre o desempenho observado.

O Backtracking seleciona células em ordem linear e tenta valores de 1 a 9 sem ordenação, sem executar nenhuma computação adicional por nó além da validação do candidato corrente. O Branch and Bound incorpora a heurística MRV para seleção de variável e uma função de bound que poda ramos onde alguma célula vazia ficou sem candidatos válidos após cada atribuição. O Greedy combina MRV com o critério LCV para ordenação dos candidatos, preferindo valores que menos restringem as células vizinhas ainda não preenchidas. A Programação Dinâmica aplica memoização sobre uma estrutura de busca com MRV, serializando o estado do tabuleiro a cada nó e evitando a reexploração de estados já provados como sem solução.

Técnica	Seleção de variável	Ordenação de candidatos	Mecanismo adicional	Custo por nó
Backtracking	Linear	Sem ordenação	Nenhum	O(1)
Branch and Bound	MRV	Sem ordenação	Função de bound	O(n)
Greedy	MRV	LCV	Nenhum	O(n)
Programação Dinâmica	MRV	Sem ordenação	Memoização	O(n)

Tabela 6 - Comparação das características das técnicas implementadas.

Os resultados experimentais confirmam que a redução no número de nós visitados proporcionada pelas heurísticas vem acompanhada de um custo por nó maior, e que o desempenho relativo de cada técnica depende do equilíbrio entre esses dois fatores para a instância específica sendo resolvida. Nas instâncias testadas, o Backtracking foi o mais rápido em tempo absoluto nas instâncias média e difícil, enquanto o Greedy apresentou o melhor equilíbrio entre redução de nós e overhead por nó.

5.1.2. Complexidade Computacional

A tabela a seguir consolida as complexidades de tempo e espaço dos quatro algoritmos implementados:

Técnica	Complexidade de tempo	Complexidade de Espaço
Backtracking	$O(9^n)$	O(n)
Branch and Bound	O(V_bb x n)	O(n)
Greedy	O(V_g x n)	O(n)
Programação Dinâmica	O(V_dp x n)	O(n + s)

Tabela 7 - Complexidade computacional dos algoritmos.

Todos os algoritmos compartilham complexidade espacial O(n) no componente dominante, que é a pilha de recursão, com profundidade máxima igual ao número de células vazias no tabuleiro inicial, sempre menor ou igual a 81. O modelo de dados, composto pelas matrizes int[9][9] e boolean[9][9] de SudokuBoard, ocupa espaço constante O(1) e todos os algoritmos operam in-place, sem criar cópias do tabuleiro entre chamadas recursivas.

A Programação Dinâmica introduz um componente adicional S referente ao conjunto de estados mortos, cujo tamanho depende do número de estados distintos explorados e descartados durante a busca. No pior caso esse conjunto pode crescer significativamente, tornando a complexidade espacial da Programação Dinâmica a maior entre os quatro algoritmos na prática, ainda que assintoticamente permaneça linear no número de estados visitados.

No que diz respeito ao tempo, o limite superior teórico de todos os algoritmos é $O(9^n)$, pois nenhuma heurística ou função de bound elimina ramos de forma assintoticamente garantida para todas as instâncias possíveis. Na prática, os algoritmos com MRV visitam uma fração muito menor do espaço de busca, expressa pelo fator V nas notações da tabela. O Backtracking tem custo O(1) por nó, enquanto os demais pagam O(n) por nó devido às varreduras adicionais do tabuleiro. O desempenho relativo de cada técnica depende portanto do produto entre o número de nós visitados e o custo por nó, conforme discutido na seção de resultados.

5.1.3. Eficiência e Desempenho

A análise de eficiência dos algoritmos deve considerar dois fatores em conjunto: o número de nós visitados, que reflete a qualidade da estratégia de busca, e o tempo de execução absoluto, que incorpora também o custo computacional por nó. Os resultados obtidos demonstram que esses dois fatores podem apontar em direções opostas, tornando a comparação entre as técnicas mais nuançada do que uma simples contagem de nós visitados sugeriria.

Em todas as três instâncias testadas, o Backtracking visitou significativamente mais nós do que os demais algoritmos, registrando 70, 86 e 85 nós nas instâncias fácil, média e difícil respectivamente, contra 20, 29 e valores

entre 42 e 53 para os algoritmos com heurística. Apesar disso, o Backtracking foi o algoritmo mais rápido em tempo absoluto nas instâncias média e difícil, com médias de 0,060 ms e 0,053 ms. Esse resultado decorre diretamente do custo O(1) por nó do Backtracking: sem heurísticas de seleção de variável ou mecanismos adicionais de poda, cada nó é processado com custo mínimo, e a soma desses custos baixos superou o benefício da redução de nós proporcionada pelas técnicas mais sofisticadas nas instâncias utilizadas.

O Greedy apresentou o melhor equilíbrio entre as duas dimensões de eficiência. Nas três instâncias, registrou o menor ou segundo menor número de nós visitados entre os algoritmos com heurística, o menor número de backtracks na instância difícil (3, contra 14 do Branch and Bound e da Programação Dinâmica), e os menores desvios padrão de tempo entre os algoritmos com MRV, indicando comportamento consistente entre execuções. Esse resultado sugere que a combinação de MRV com LCV produz um caminho de busca particularmente estável, evitando tanto a exploração desnecessária quanto a variabilidade introduzida por fatores externos como coleta de lixo da JVM.

O Branch and Bound visitou mais nós do que o Greedy nas três instâncias e foi mais lento em tempo absoluto, o que indica que a função de bound, embora conceitualmente mais poderosa do que o LCV, não compensou seu overhead adicional nas instâncias testadas. A Programação Dinâmica apresentou o maior desvio padrão na instância difícil (0,565 ms), sugerindo que o custo de serialização e consulta ao conjunto de estados mortos introduz variabilidade dependente do caminho de busca percorrido em cada execução.

De forma geral, os resultados indicam que, para instâncias de Sudoku com espaço de busca moderado, o custo por nó é o fator dominante no tempo de execução total. A vantagem assintótica dos algoritmos com heurística tende a se manifestar em instâncias com espaço de busca muito maior, onde a redução no número de nós visitados passa a dominar o overhead por nó.

5.1.4. Qualidade da Solução

No contexto do Sudoku, a qualidade da solução é definida de forma binária: ou o tabuleiro está completamente preenchido e todas as restrições de unicidade

por linha, coluna e subgrade 3×3 são satisfeitas, ou a solução é inválida. Não há espaço para soluções parciais ou aproximadas, o que torna a verificação de corretude direta e objetiva.

Todos os quatro algoritmos produziram soluções válidas e completas nas três instâncias testadas, confirmando que as diferentes estratégias de busca, seleção de variável e poda não comprometem a corretude do resultado final. Esse comportamento era esperado pela análise teórica: as heurísticas MRV e LCV alteram apenas a ordem de exploração da árvore de busca, sem eliminar ramos que contenham soluções válidas, e a função de bound do Branch and Bound poda exclusivamente estados onde alguma célula vazia ficou sem candidatos, situação que por definição não pode levar a nenhuma solução válida.

A verificação formal da qualidade das soluções é realizada automaticamente pelos testes JUnit 5 por meio do método tabuleiroCompletoEValido da classe TestUtils, que percorre todas as 81 células da solução retornada, confirma que cada valor está no intervalo [1, 9] e invoca SudokuValidator.isValid célula a célula após zerá-la temporariamente, evitando falso conflito da célula consigo mesma. Essa verificação é executada para os quatro algoritmos nas três instâncias de dificuldade em BacktrackingSolverTest, BranchAndBoundSolverTest, GreedySolverTest, DynamicProgrammingSolverTest e SolverComparisonTest, garantindo que nenhuma solução produzida viole as regras do jogo.

5.1.5. Aplicabilidade

A escolha do algoritmo mais adequado depende do contexto de uso e das características das instâncias a serem resolvidas.

O Backtracking é indicado quando simplicidade de implementação e baixo footprint computacional são prioritários. Por não depender de estruturas auxiliares nem de varreduras adicionais por nó, é o mais adequado para ambientes com restrições severas de recursos, e os resultados obtidos confirmam que seu baixo custo por nó compensa o maior número de nós visitados em instâncias com espaço de busca moderado.

O Greedy é a opção mais equilibrada entre os algoritmos com heurística. A combinação de MRV com LCV produziu o menor número de backtracks e o

comportamento mais estável entre execuções nas instâncias testadas, sem introduzir estruturas auxiliares de custo elevado, sendo a escolha mais indicada quando se busca eficiência e consistência sem abrir mão de heurísticas de qualidade.

O Branch and Bound é indicado quando se deseja poda sistemática em dois níveis: na seleção da célula mais restrita via MRV e na verificação do impacto de cada atribuição sobre as células vizinhas via função de bound. Essa dupla camada de poda é especialmente útil em instâncias onde atribuições incorretas propagam conflitos rapidamente pelo tabuleiro.

A Programação Dinâmica é mais vantajosa quando o espaço de busca contém um número expressivo de estados repetidos. No Sudoku padrão 9×9, a estrutura da busca com MRV limita a ocorrência desses estados, reduzindo o benefício prático da memoização e tornando o custo de serialização e consulta ao conjunto de estados mortos o fator dominante no desempenho observado.

5.1.6. Vantagens e Desvantagens

Técnica	Vantagens	Desvantagens
Backtracking	Implementação simples; menor custo por nó; baixo uso de memória; comportamento previsível	Maior número de nós visitados; sem detecção precoce de conflitos; sensível a instâncias com espaço de busca vasto
Branch and Bound	Poda em dois níveis via MRV e função de bound; detecta inviabilidade antes de descer na recursão	Custo O(n) por nó; overhead da função de bound pode superar o ganho de poda em instâncias simples
Greedy	Melhor equilíbrio entre redução de nós e custo por nó; comportamento estável entre execuções; menor número de backtracks nas instâncias testadas	Custo adicional do LCV por nó; ordenação por selection sort; sem mecanismo de poda antecipada como o bound
Programação Dinâmica	Evita reexploração de estados já provados como sem solução; eficaz quando estados repetidos são frequentes	Custo de serialização e consulta ao conjunto de estados mortos por nó; maior variabilidade de tempo; benefício limitado no Sudoku padrão devido à baixa recorrência de estados repetidos

Tabela 8 - Vantagens e Desvantagens das técnicas implementadas.

5.1.7. Síntese Comparativa

Os quatro algoritmos implementados resolveram corretamente todas as instâncias testadas, confirmando que as diferentes estratégias de busca não comprometem a corretude do resultado. A distinção entre eles reside inteiramente no desempenho: número de nós visitados, número de backtracks e tempo de execução.

O principal achado dos experimentos é que o algoritmo mais simples, o Backtracking, foi o mais rápido em tempo absoluto nas instâncias média e difícil, enquanto os algoritmos com heurística visitaram consistentemente menos nós. Esse resultado evidencia que, para as instâncias testadas, o custo por nó foi o fator dominante no tempo total de execução, superando o benefício da redução no número de nós explorados. Isso não invalida a superioridade assintótica dos algoritmos com heurística, mas indica que essa vantagem se manifesta em instâncias com espaço de busca significativamente maior do que o das instâncias utilizadas.

Entre os algoritmos com heurística, o Greedy apresentou o melhor desempenho geral, combinando redução expressiva de nós visitados com o menor overhead por nó do grupo e o comportamento mais estável entre execuções. O Branch and Bound e a Programação Dinâmica introduziram mecanismos adicionais de poda e memoização que, nas instâncias testadas, não compensaram seus custos por nó mais elevados.

A conclusão prática é que a escolha do algoritmo deve considerar não apenas a complexidade assintótica, mas o produto entre o custo por nó e o número de nós visitados para a classe de instâncias em questão. Em instâncias de Sudoku padrão 9×9 com espaço de busca moderado, estratégias simples e de baixo custo por nó tendem a ser competitivas ou superiores a técnicas mais sofisticadas.

6. CONCLUSÃO

Este trabalho apresentou a implementação e a análise comparativa de quatro técnicas de projeto de algoritmos aplicadas à resolução do Sudoku: Backtracking, Branch and Bound, Greedy e Programação Dinâmica. O Sudoku, por ser um problema de satisfação de restrições com estrutura bem definida e espaço de busca finito, constitui um ambiente controlado e expressivo para a avaliação prática dessas técnicas, permitindo isolar o impacto de cada decisão de projeto sobre o desempenho observado.

A análise teórica demonstrou que todos os algoritmos compartilham o mesmo limite superior assintótico de $O(9^n)$ no pior caso, diferindo na constante prática determinada pelo produto entre o custo por nó e o número de nós efetivamente visitados. Os experimentos confirmaram essa relação de forma concreta: o Backtracking foi o mais rápido em tempo absoluto nas instâncias média e difícil, evidenciando que o overhead introduzido pelas heurísticas pode superar seu benefício em instâncias com espaço de busca moderado. O Greedy destacou-se como a técnica de melhor equilíbrio, combinando redução expressiva de nós com comportamento estável e custo por nó competitivo.

Mais do que identificar o algoritmo vencedor, os experimentos ilustram uma lição fundamental do projeto de algoritmos: complexidade assintótica e desempenho prático são dimensões distintas e igualmente importantes. Um algoritmo teoricamente superior pode ser superado na prática por um mais simples quando as constantes ocultas pela notação assintótica são desfavoráveis para a classe de instâncias em questão. Essa tensão entre teoria e prática é o que torna o projeto de algoritmos uma disciplina rica e não trivial, e os resultados obtidos neste trabalho constituem uma ilustração concreta e direta desse princípio.

7. REFERÊNCIAS

DA CRUZ, Lucas Ribeiro et al. **RECONHECIMENTO DE DÍGITOS EM TABULEIROS DE SUDOKU COM VISÃO COMPUTACIONAL E RESOLUÇÃO POR FORÇA BRUTA**. REVISTA FOCO , v. 6, pág. e8944-e8944, 2025. Dispon[ivel em: https://doi.org/10.54751/revistafoco.v18n6-206. Acesso em: 12/03/2026.

EFFENDI, Rustam; GUNAWAN, Indra; EFENDI, Yasin. **Projeto de Software para Conclusão do Jogo Sudoku com Algoritmo Branch and Bound.** In: 1ª Conferência Internacional Multidisciplinar sobre Educação, Tecnologia e Engenharia (IMCETE 2019) . Atlantis Press, 2020. p. 126-130. Disponível em: https://www.atlantis-press.com/proceedings/imcete-19/125935522. Acesso em: 12/03/2026.

G1. **Sudoku.** G1 – Portal de Notícias da Globo. Disponível em: https://g1.globo.com/jogos/sudoku/. Acesso em: 11/03/2026.

TAKANO, Kevin; DE FREITAS, Rosiane; DE SÁ, Vinícius. O jogo de lógica sudoku: modelagem teórica, np-completude e estratégias algorítmicas exatas e heurísticas. In: **Concurso de Trabalhos de Iniciação Científica da SBC (CTIC-SBC)**. SBC, 2015. p. 71-80.
Disponível em: https://sol.sbc.org.br/index.php/ctic/article/view/10020. Acesso em: 31/03/2026.

Link do repositório: https://github.com/eduardanb/sudoku-solver-taal.git